

金融软工 实验五

0、接口文档

1、项目前端页面展示

2、功能代码展示

3、代码逻辑

3.1 后端 get_user_info函数实现逻辑：

3.2 前端404页面代码核心逻辑

1. 组件结构设计

2. 视觉呈现方案

3. 信息展示策略

4. 响应式布局机制

5. 交互导航实现

6. 色彩系统应用

3.3 前端获取用户信息代码核心逻辑

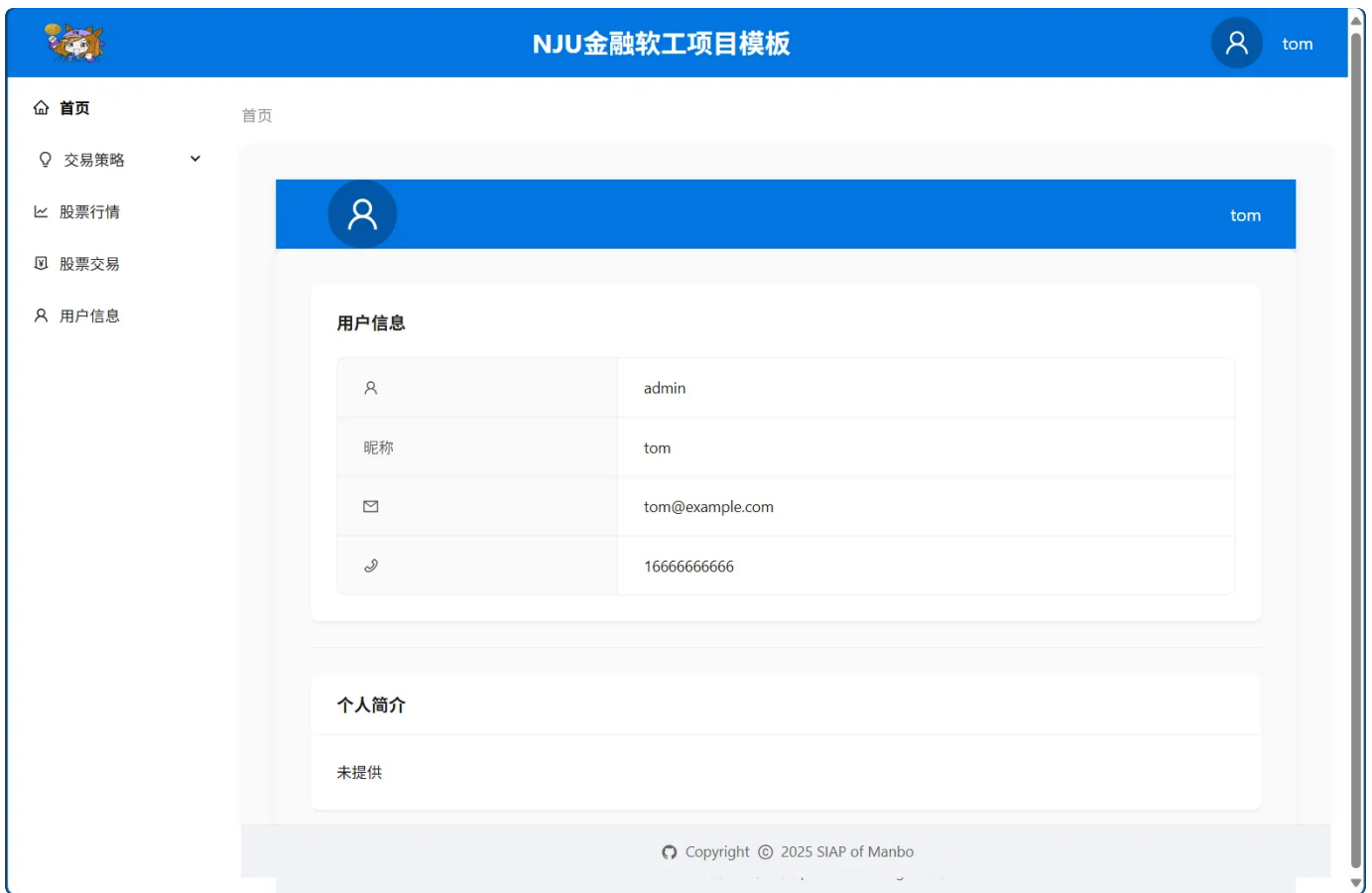
4、小组分工

0、接口文档

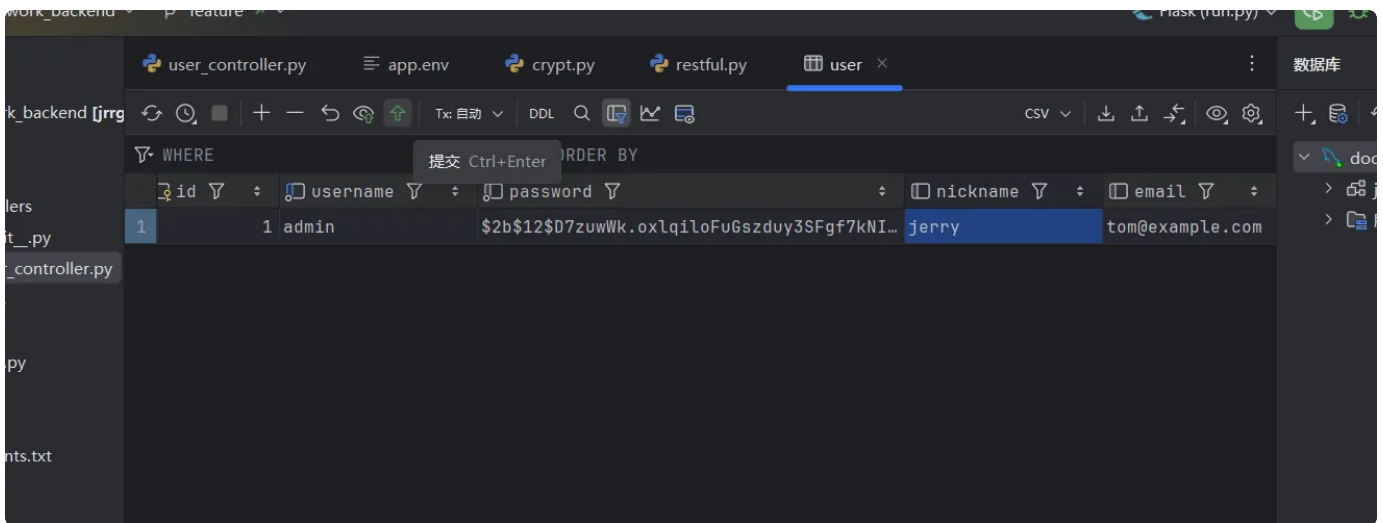
文档是一个语雀文档，地址：[第一组接口文档](#)

1、项目前端页面展示

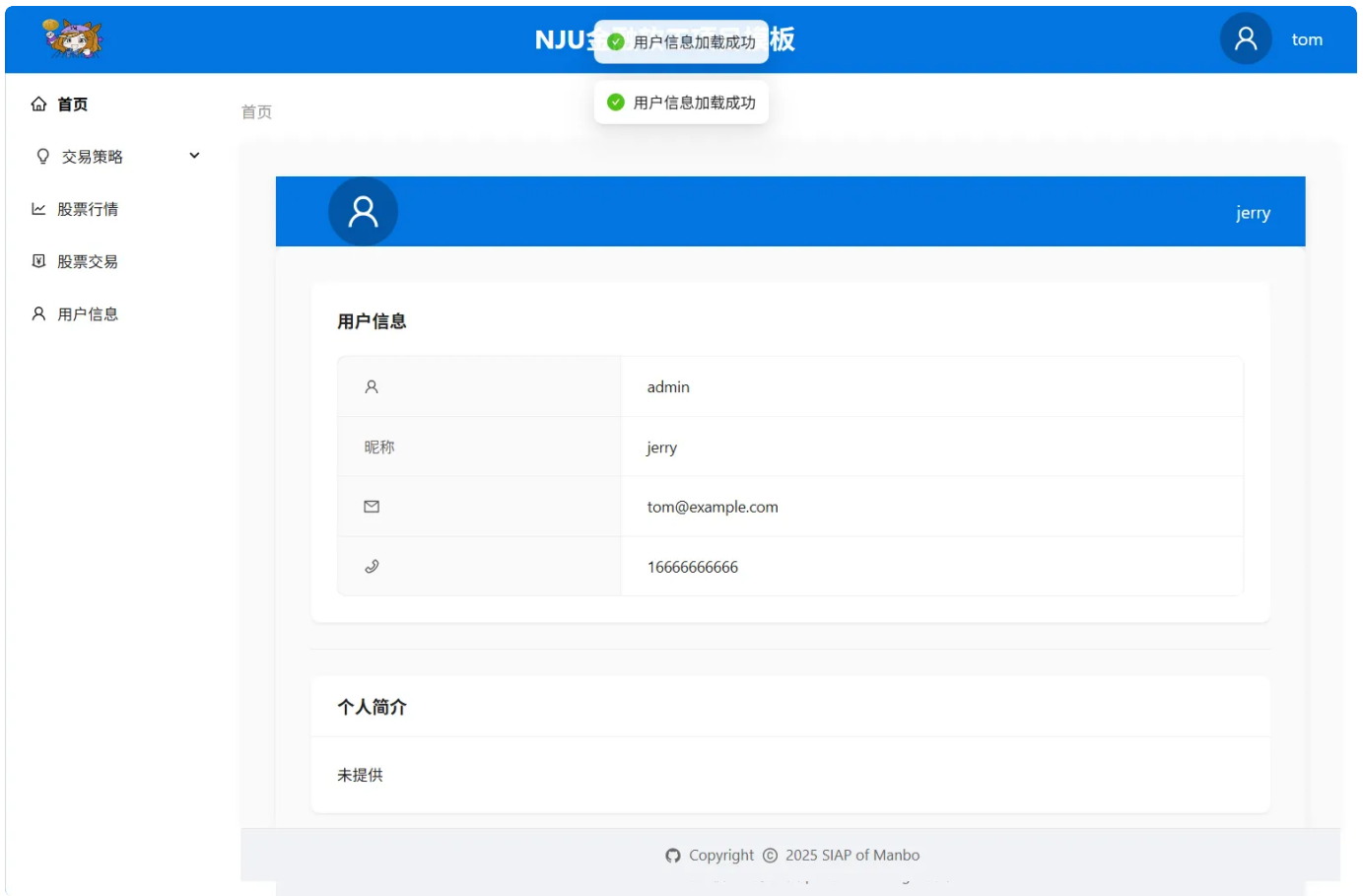
- 数据库修改前用户信息页面



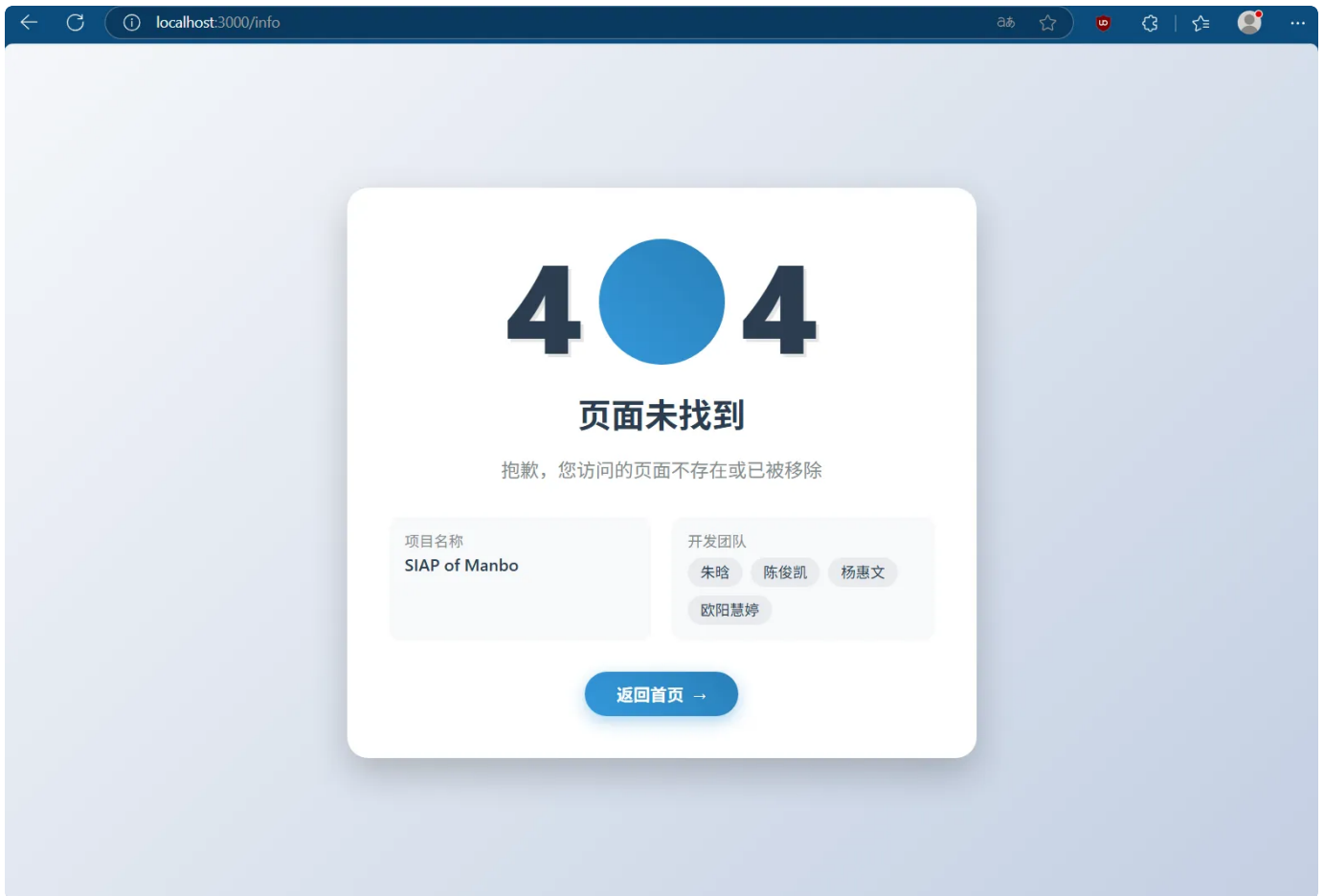
- 后端数据库昵称修改——nickname: tom→jerry



- 数据库修改后用户信息页面
- Note: 修改数据库中的nickname会使得用户信息页面的昵称发生更改, 但右上角的昵称由localStorage直接读取, 故未发生变更。而实际应用中并不会出现在数据库中直接修改用户信息的情况。



- 404页面展示

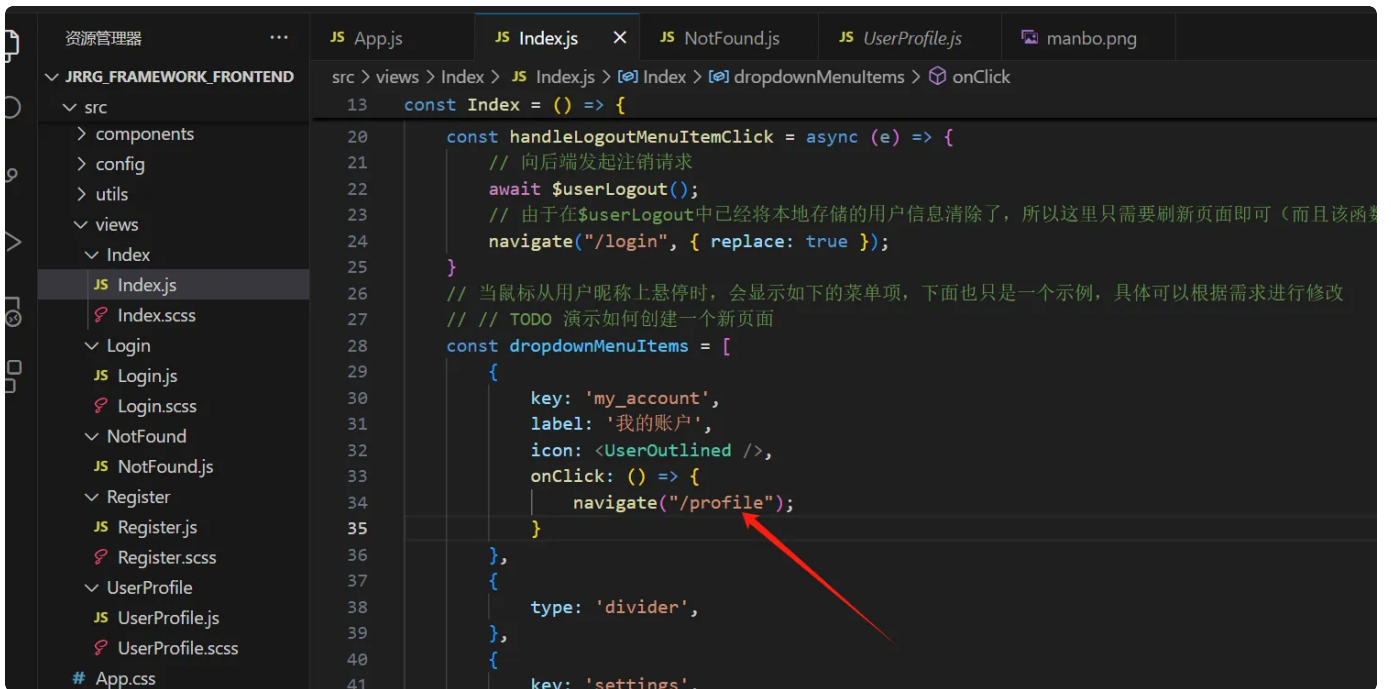


2、功能代码展示

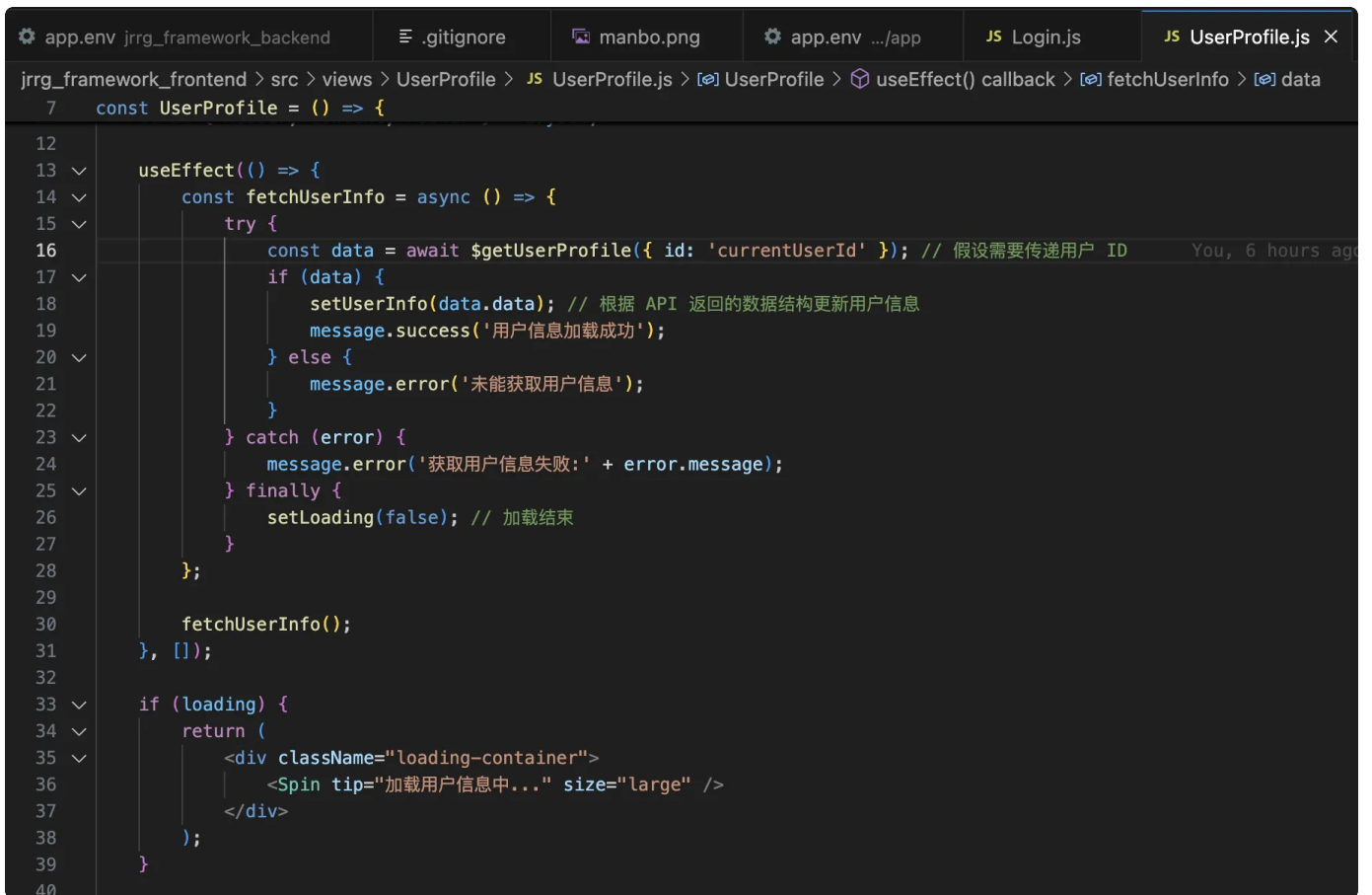
- 用户信息展示页面设计

```
7   const UserProfile = () => {  
45     return (  
46       <Layout className="user-profile-layout">  
47         {/* 头部 */}  
48         <Header className="header">  
49           <Avatar  
50             className="avatar"  
51             size={64}  
52             icon={<UserOutlined />}  
53             src={userInfo.avatar || null}  
54           />  
55           <span className="nickname">{userInfo.nickname}</span>  
56         </Header>  
57         {/* 主体内容 */}  
58         <Content className="content">  
59           <Card className="user-info-card" bordered={false}>  
60             <Descriptions  
61               title="用户信息"  
62               bordered  
63               column={1}  
64               layout="horizontal"  
65             >  
66               <Descriptions.Item label={<UserOutlined />}>  
67                 {userInfo.username || '未提供'}  
68               </Descriptions.Item>  
69               <Descriptions.Item label="昵称">  
70                 {userInfo.nickname || '未提供'}  
71               </Descriptions.Item>  
72               <Descriptions.Item label={<MailOutlined />}>  
73                 {userInfo.email || '未提供'}  
74               </Descriptions.Item>  
75               <Descriptions.Item label={<PhoneOutlined />}>  
76                 {userInfo.phone || '未提供'}  
77               </Descriptions.Item>  
78             </Descriptions>  
79           </Card>  
80  
81           <Divider />  
82  
83           <Row gutter={[16, 16]}>  
84             <Col span={24}>
```

- 用户信息页面调用方法



```
src > views > Index > JS Index.js > [?] Index > [?] dropdownMenuItems > [?] onClick
13  const Index = () => {
20      const handleLogoutMenuItemClick = async (e) => {
21          // 向后端发起注销请求
22          await $userLogout();
23          // 由于在$userLogout中已经将本地存储的用户信息清除了，所以这里只需要刷新页面即可（而且该函数
24          navigate("/login", { replace: true });
25      }
26      // 当鼠标从用户昵称上悬停时，会显示如下的菜单项，下面也只是一个示例，具体可以根据需求进行修改
27      // // TODO 演示如何创建一个新页面
28      const dropdownMenuItems = [
29          {
30              key: 'my_account',
31              label: '我的账户',
32              icon: <UserOutlined />,
33              onClick: () => {
34                  navigate("/profile");
35              }
36          },
37          {
38              type: 'divider',
39          },
40          {
41              key: 'settings',
```



```
app.env jrrg_framework_backend | .gitignore | manbo.png | app.env .../app | JS Login.js | JS UserProfile.js X
jrrg_framework_frontend > src > views > UserProfile > JS UserProfile.js > [?] UserProfile > [?] useEffect() callback > [?] fetchUserInfo > [?] data
7  const UserProfile = () => {
12
13      useEffect(() => {
14          const fetchUserInfo = async () => {
15              try {
16                  const data = await $getUserProfile({ id: 'currentUserId' }); // 假设需要传递用户 ID
17                  if (data) {
18                      setUserInfo(data.data); // 根据 API 返回的数据结构更新用户信息
19                      message.success('用户信息加载成功');
20                  } else {
21                      message.error('未能获取用户信息');
22                  }
23              } catch (error) {
24                  message.error('获取用户信息失败: ' + error.message);
25              } finally {
26                  setLoading(false); // 加载结束
27              }
28          };
29
30          fetchUserInfo();
31      }, []);
32
33      if (loading) {
34          return (
35              <div className="loading-container">
36                  <Spin tip="加载用户信息中..." size="large" />
37              </div>
38          );
39      }
40  }
```

- 后端新增controller函数，获取并返回用户信息

- logo导入

```

src > views > Index > JS Index.js > [?] Index > [?] dropdownMenuItems > [?] onClick
1  import { HomeOutlined, LineChartOutlined, MoneyCollectOutlined, BulbOutlined, EditOutlined, Rollb
2  import { Breadcrumb, Layout, Menu, Avatar, Tooltip, Dropdown } from 'antd';
3  import React from 'react';
4  import './Index.scss';
5  // 在create-react-app中, 有两种方式引入图片资源: 1. 对于存放在public目录下的资源, 可以直接通过相对路径引入
6  import Logo from '../../assets/img/manbo.png';
7  import { Outlet } from 'react-router-dom';
8  import { $userLogout } from '../../api/userApi';
9  import { useNavigate } from 'react-router-dom';
10
11

```

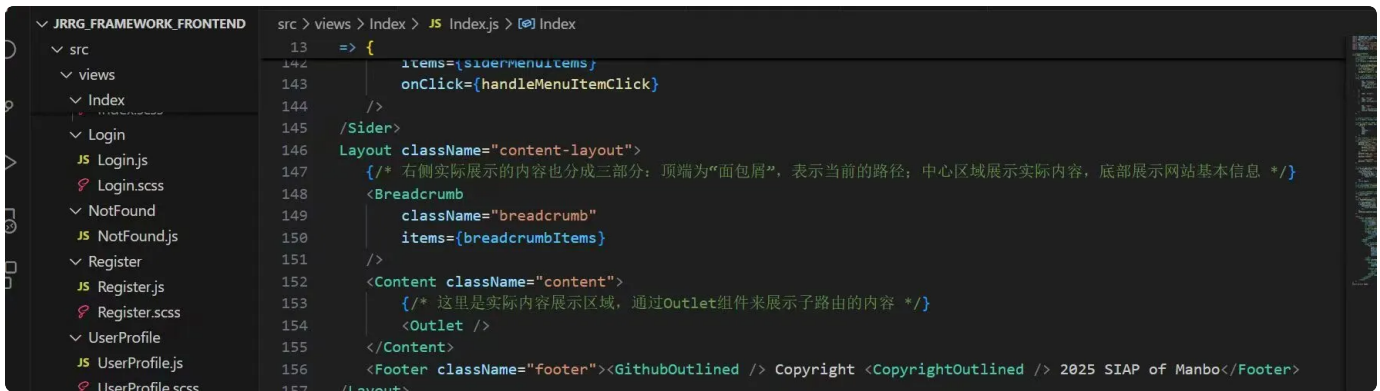
- 前端api对用户信息的访问

```

jrrg_framework_frontend > src > api > JS userApi.js > [?] $getUserProfile
47  * 用户注册, 根据传入的用户数据, 注册一个新用户
48  * @param {object} registerData
49  */
50  export const $userRegister = async (registerData) => {
51      // 对于这种不需要返回结果的请求, 我们只需要根据状态码得到本次请求的结果, 那么
52      await http.post("/api/user/register", registerData);
53  }
54
55  /**
56  * 获取用户信息, 并加以展示
57  * @param {object} userProfileData
58  * */
59  export const $getUserProfile = async (userProfileData) => {
60      // 这里的userProfileData是一个对象, 包含了用户的id和token
61      let res = null; // 在函数作用域中声明
62      try{
63          res=await http.get("/api/user/query", userProfileData);
64      } catch (error) { // 这里简单处理, 只是打印错误信息
65          console.error("获取用户信息失败", error);
66      } finally {
67          // 保证一定执行
68          return res;
69      }
70  }
71
72

```

- 修改copyright



3、代码逻辑

3.1 后端 `get_user_info` 函数实现逻辑：

- 身份验证：使用 `@jwt_required()` 装饰器，确保请求携带合法的 JWT Token，即用户已登录。
- 提取用户ID：通过 `get_jwt_identity()` 从 JWT Token 中获取当前用户的 ID。
- 查询数据库：使用 SQLAlchemy 查询 `User` 表中与该 ID 对应的用户记录。
- 返回用户信息：将该用户的关键信息（如 username、nickname、email、phone）封装成字典，使用统一的 `utils.success()` 方法返回给前端。
- 异常处理：如果用户不存在，则返回 `404` 错误信息。

3.2 前端404页面代码核心逻辑

1. 组件结构设计

1

<NotFound>

2

├ 错误代码展示区（静态404数字）

3

├ 文字说明区（标题+描述文本）

4

├ 信息展示区

5

│├ 项目信息卡（展示项目名称）

6

│└ 团队信息卡（展示成员列表）

7

└ 操作引导区（返回首页链接）

采用垂直流式布局，遵循用户阅读习惯（信息层级从上到下）

2. 视觉呈现方案

```
1  ▼ errorCode: {  
2    fontSize: "120px",           // 强调性超大字号  
3    fontWeight: "900",           // 厚重字重增强存在感  
4    color: "#2c3e50",           // 深色系提升可读性  
5    display: "flex"              // 数字水平居中排列  
6  }
```

通过字号阶梯 (120px → 32px → 18px) 建立清晰的信息层级

3. 信息展示策略

双栏卡片布局

```
1  ▼ infoContainer: {  
2    display: "flex",              // 弹性容器  
3    gap: "20px",                  // 元素间距控制  
4    justifyContent: "space-between" // 等宽分布  
5  }  
6  
7  // 项目信息样式  
8  ▼ projectName: {  
9    fontSize: "16px",             // 主次文本大小对比  
10   fontWeight: "600"              // 重点信息加粗  
11 }  
12  
13 // 团队成员标签  
14 ▼ member: {  
15   borderRadius: "20px",          // 胶囊状标签设计  
16   background: "#e9ecef"          // 浅底突出文字  
17 }
```

使用对比色 (#2c3e50 vs #7f8c8d) 区分标签与内容

4. 响应式布局机制

```

1  container: {
2    minHeight: "100vh",      // 全视口高度适配
3    padding: "20px"          // 安全边距
4  }
5
6  card: {
7    maxWidth: "600px",       // 大屏内容宽度限制
8    width: "100%"            // 小屏自动填充
9  }
10
11 members: {
12   flexWrap: "wrap"          // 成员标签自动换行
13 }

```

弹性布局(flex)配合百分比宽度实现多端适配

5. 交互导航实现

```

1  // 路由链接组件
2  <Link to="/" style={styles.link}>
3    返回首页
4    <span style={styles.arrow}>&rarr;</span>
5  </Link>
6  // 按钮视觉规范
7  link: {
8    background: "linear-gradient(...)", // 渐变背景
9    borderRadius: "30px",              // 圆角胶囊造型
10   boxShadow: "0 5px 15px..."        // 层次感投影
11 }

```

通过高对比度按钮引导用户核心操作

6. 色彩系统应用

元素类型	颜色方案	设计意图
背景层	浅灰渐变	降低视觉疲劳
错误代码	#2c3e50 (深蓝灰)	强化错误标识

说明文本	#7f8c8d (中性灰)	辅助信息弱化
操作按钮	蓝系渐变	引导用户交互
信息卡背景	#f8f9fa (极浅灰)	内容区分与聚焦

该实现方案通过严谨的视觉层次、模块化信息展示和明确的交互引导，构建了一个专业且用户友好的错误提示界面，既满足功能性需求，也保持品牌调性的一致性。

3.3 前端获取用户信息代码核心逻辑

在前端 `api/userApi.js` 中添加了如下逻辑

JavaScript

```
1  /**
2   * 获取用户信息,并加以展示
3   * @param {object} userProfileData
4   */
5  export const $getUserProfile = async (userProfileData) => {
6    // 这里的userProfileData是一个对象, 包含了用户的id和token
7    let res = null; // 在函数作用域中声明
8    try{
9      res=await http.get("/api/user/query", userProfileData);
10   } catch (error) { // 这里简单处理, 只是打印错误信息
11     console.error("获取用户信息失败", error);
12   } finally {
13     // 保证一定执行
14     return res;
15   }
16 }
17
```

其发送了 `http:GET` 请求，后端将根据 `query` 字段路由到相应的处理逻辑。

同时，在 `views/UserProfile` 中调用这个方法，以实现进入页面后读取信息。

```
1  useEffect(() => {  
2      const fetchUserInfo = async () => {  
3          try {  
4              const data = await $getUserProfile({ id: 'currentUserId' }  
              ); // 假设需要传递用户 ID  
5              if (data) {  
6                  setUserInfo(data.data); // 根据 API 返回的数据结构更新用户  
              信息  
7                  message.success('用户信息加载成功');  
8              } else {  
9                  message.error('未能获取用户信息');  
10             }  
11         } catch (error) {  
12             message.error('获取用户信息失败:' + error.message);  
13         } finally {  
14             setLoading(false); // 加载结束  
15         }  
16     };  
17  
18     fetchUserInfo();  
19 }, []);
```

4、小组分工

任务内容	负责人
编写项目接口文档，完善404页面， 修改copyright	杨惠文
新建用户信息页面，展示用户信息	朱晗
新增controller，查询用户信息	陈俊凯，欧阳慧婷
编写实验文档	陈俊凯